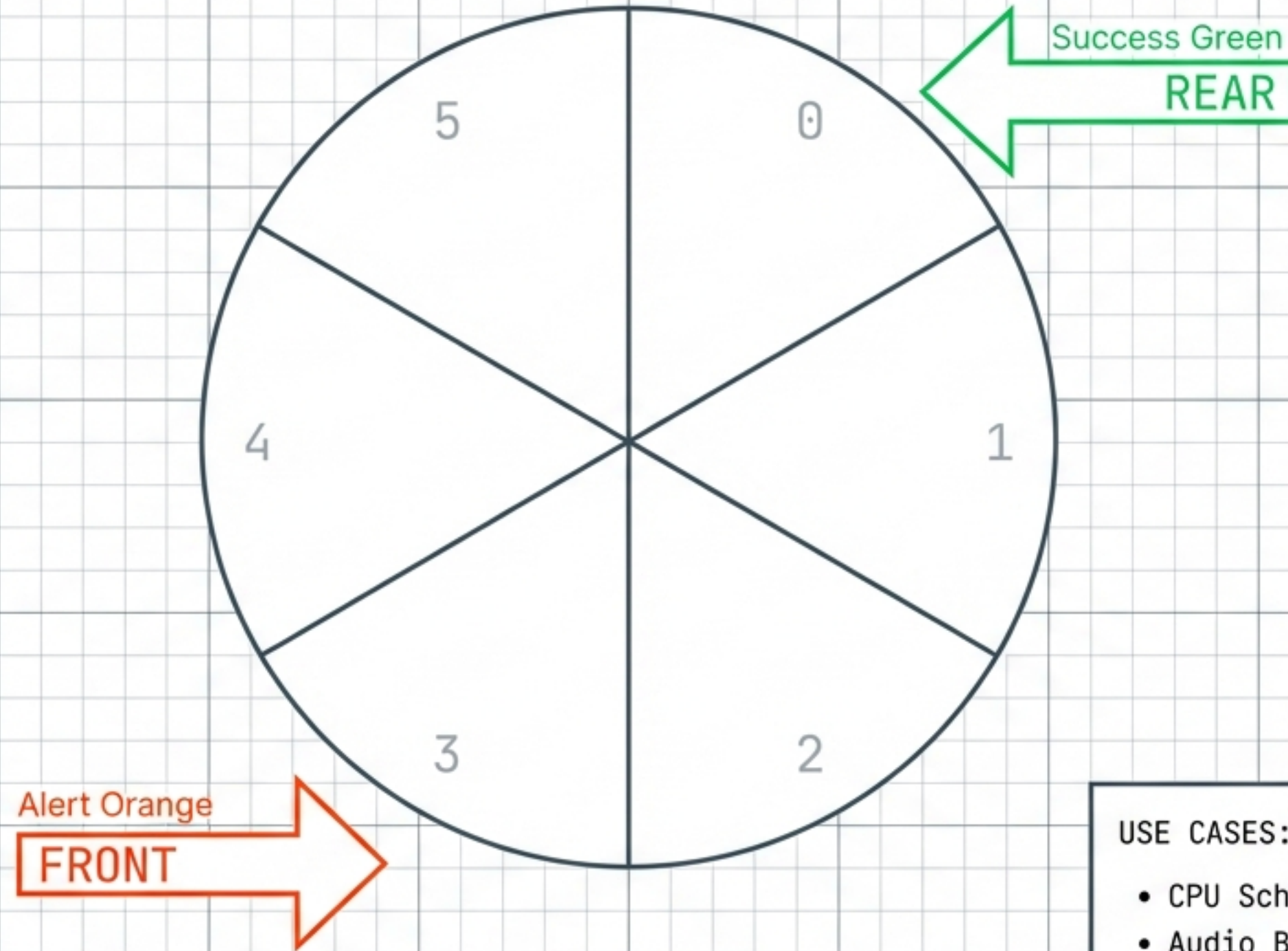


Mastering the Circular Queue

Optimization for High-Performance Systems



USE CASES:

- CPU Scheduling (Round Robin)
- Audio Buffers (Spotify)
- Traffic Shaping (AWS/Digital Ocean)

Welcome to the Top 50%

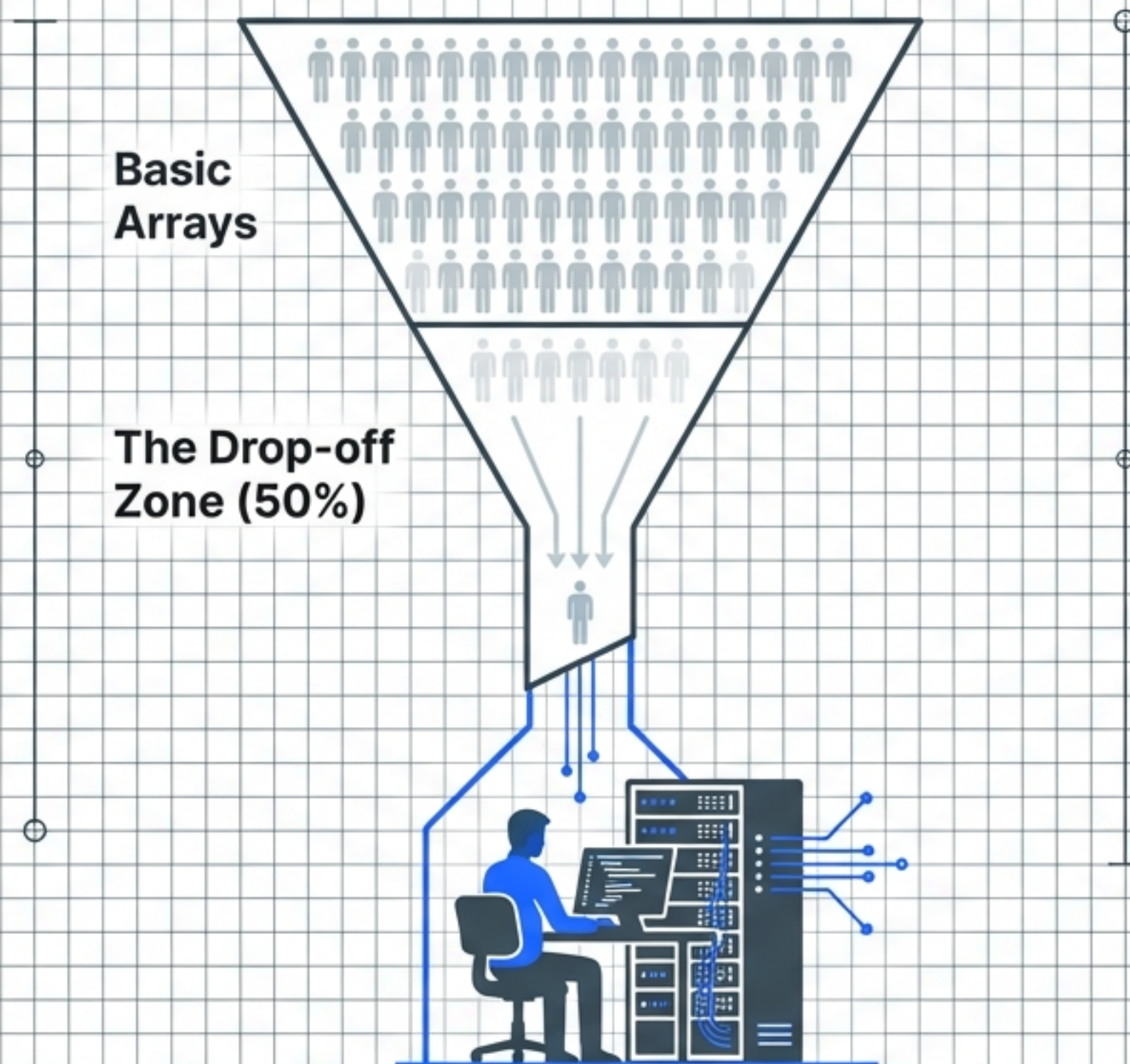
Most learners drop out before reaching Circular Queues. This is where the competition ends and real engineering begins.

Companies like Digital Ocean and Vultr rely on this logic for day-to-day operations.

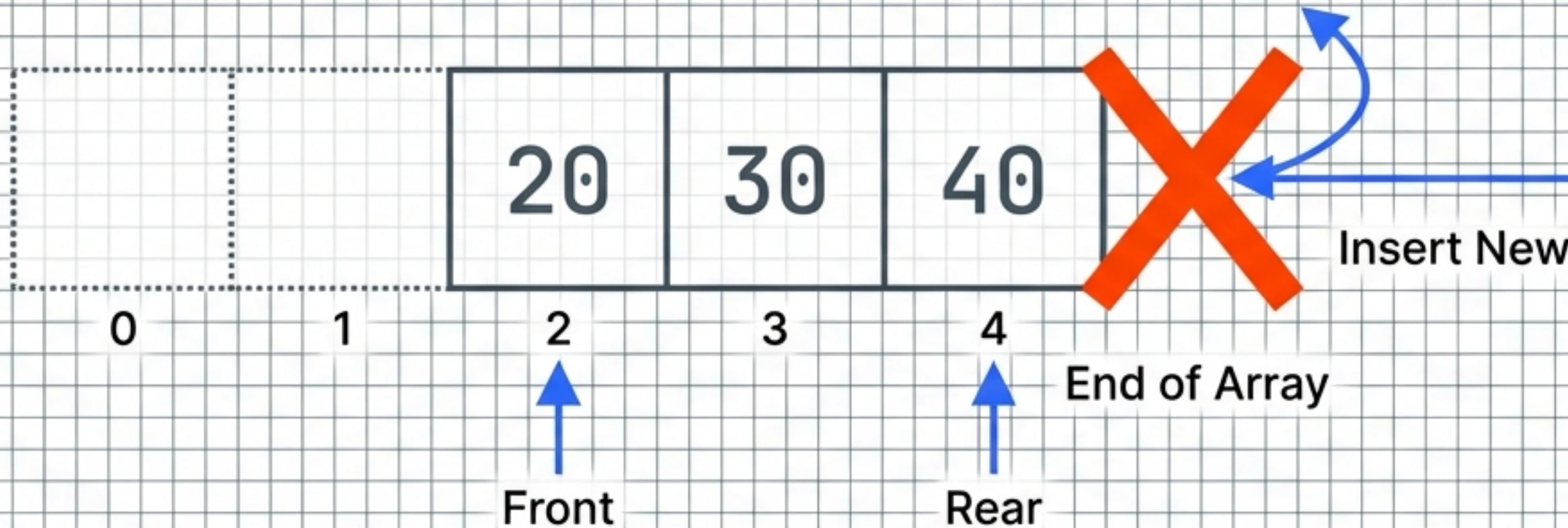
Basic Arrays

The Drop-off Zone (50%)

Without visualization (drawing on paper), this concept is impossible to master.



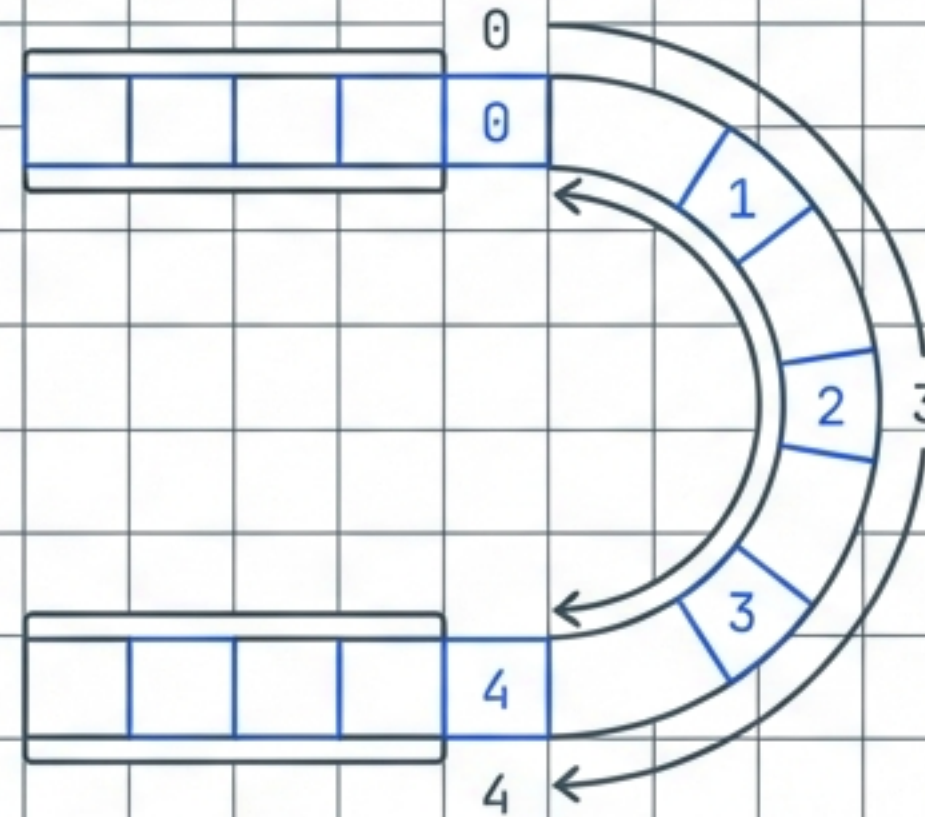
The Linear Flaw: The 'Phantom Full' State



- **THE SCENARIO:** Elements at index 0 and 1 were deleted.
- **THE ERROR:** Rear pointer has hit the limit (Size - 1).
- **THE RESULT:** Cannot insert new data despite available memory.

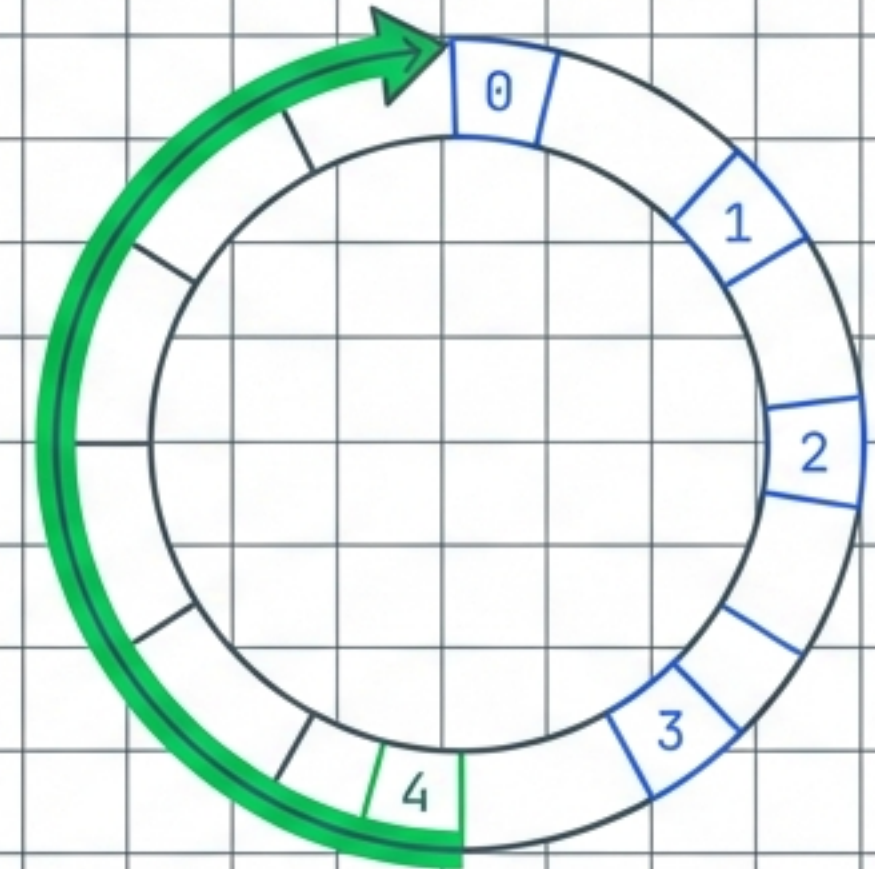
In a fixed-size array, 'Empty' doesn't always mean 'Available'.

Bending the Line



To utilize the wasted space at the front, we connect the Last Index back to the First Index.

When Rear hits the limit, the next step isn't an Error—it is Index 0.



Like a game of musical chairs or a gun cylinder. The cycle never ends, it just resets.

The Magic Operator: Modulo (%)

$$\text{Next Position} = (\text{Current} + 1) \% \text{Size}$$

How do we tell the code to loop? We use the Remainder operator.

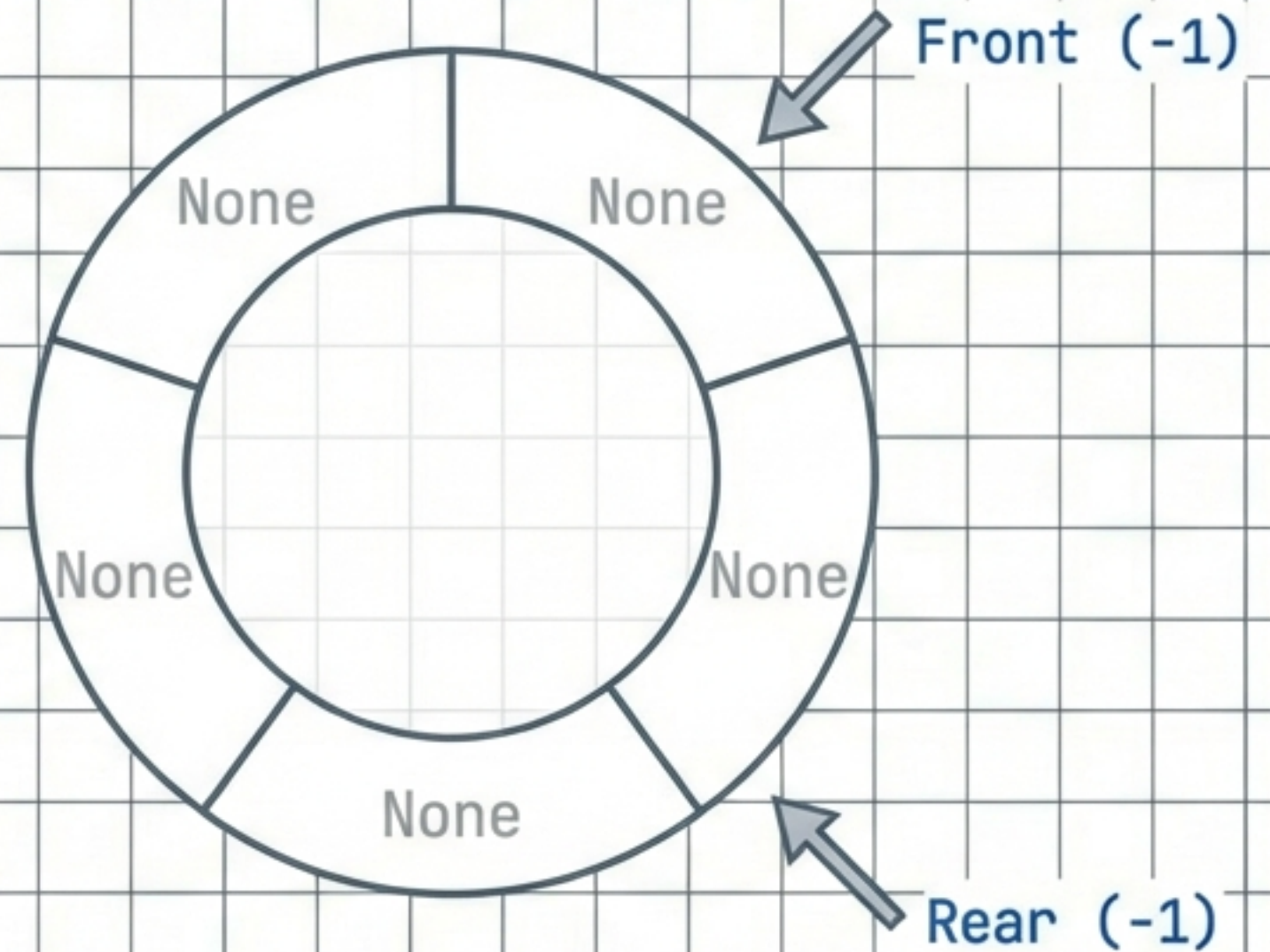
This formula acts as an automatic "Reset Button" when pointers exceed the limit.

Size = 5		
Index 3	$(3 + 1) \% 5$	Result: 4
Index 4	$(4 + 1) \% 5$	Result: 0 (RESET)
Index 5	$(5 + 1) \% 5$	Result: 1

The Blueprint (Initialization)

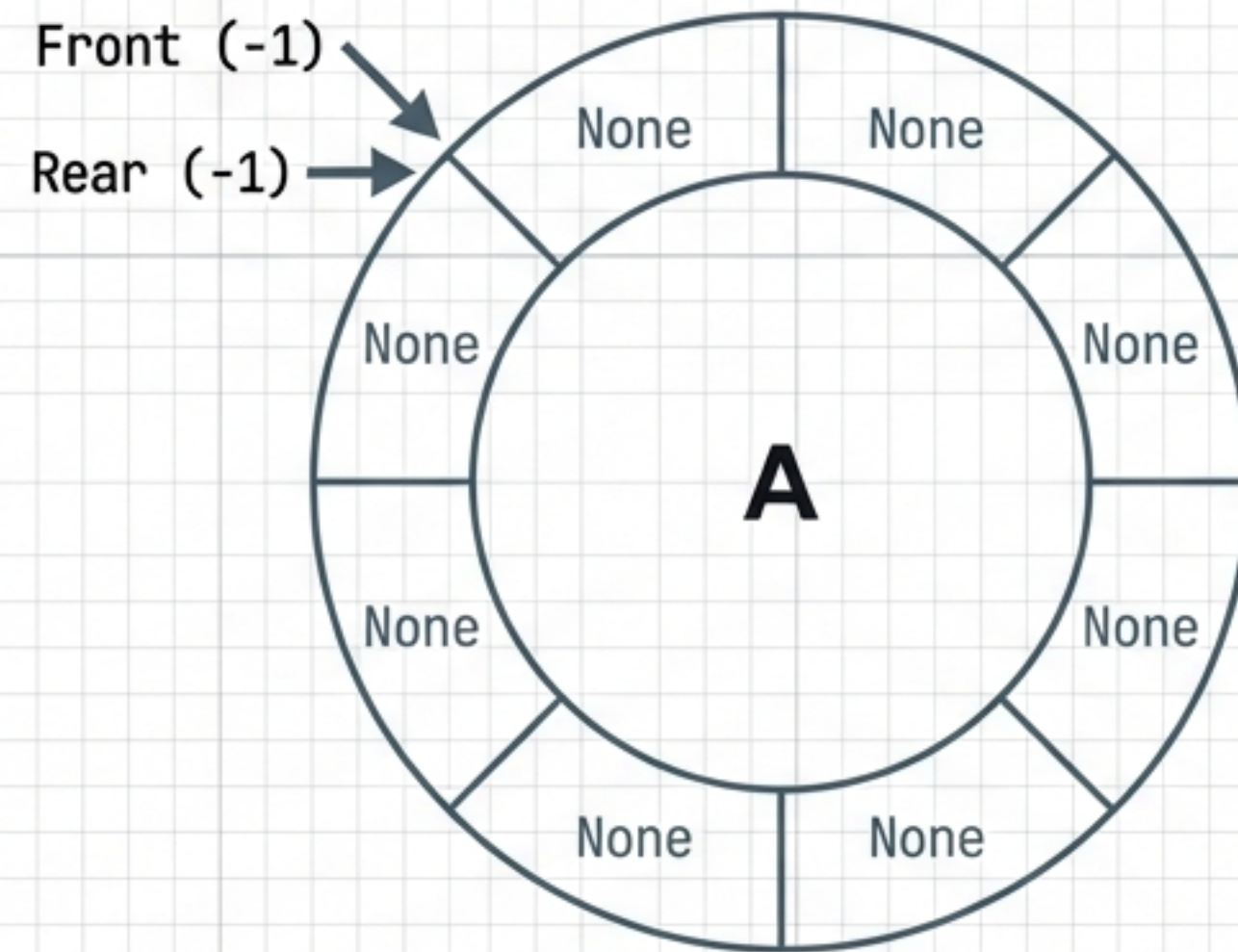
We enforce a fixed size to simulate low-level memory constraints. Start state is -1.

```
class CircularQueue:  
    def __init__(self, size):  
        self.size = size  
        self.items = [None] * size  
        self.front = -1  
        self.rear = -1
```

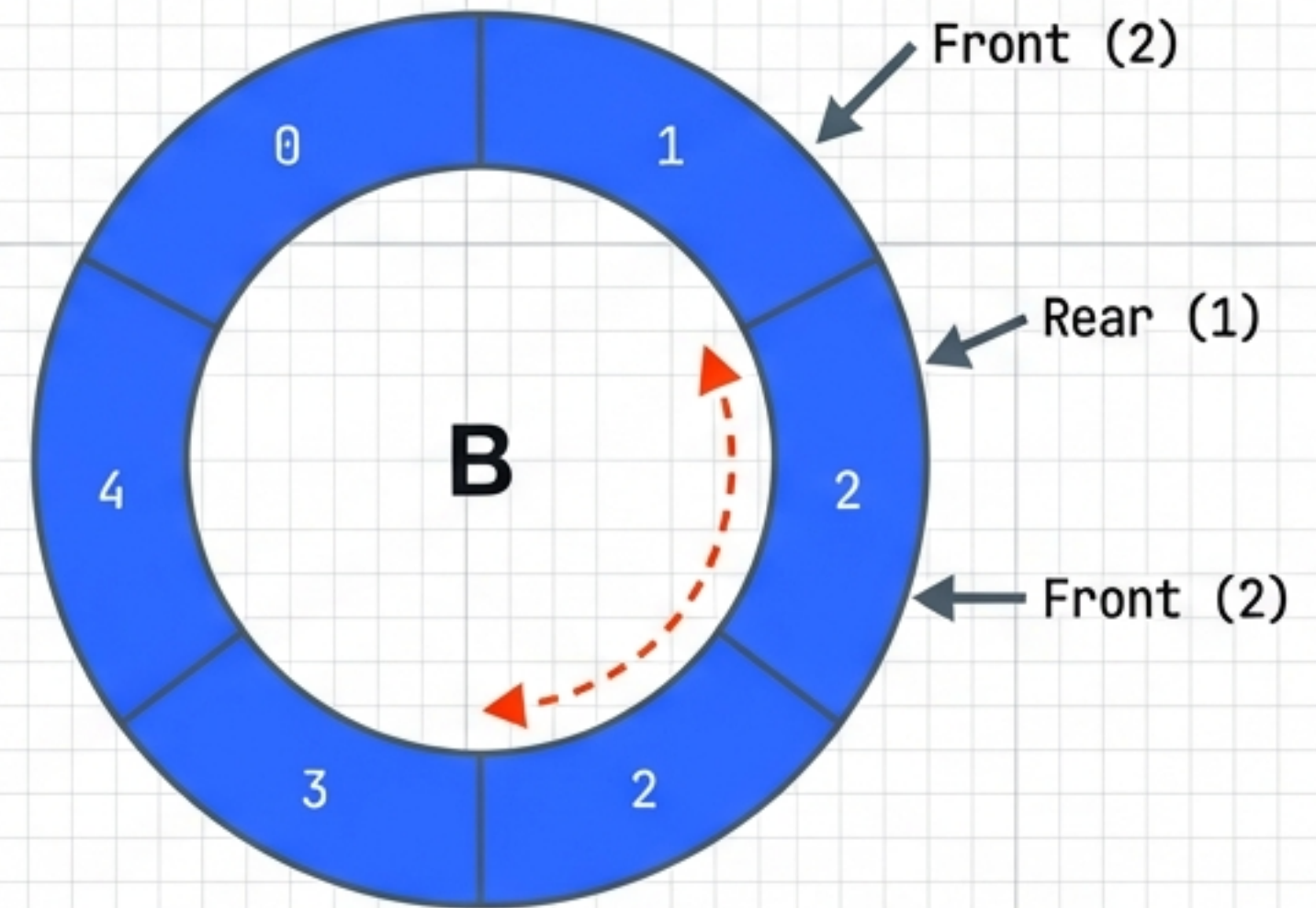


Critical States: Empty vs. Full

We must define the rules of engagement before writing the logic.
If Rear catches up to Front, the queue is at capacity.



IS EMPTY: `self.front == -1`

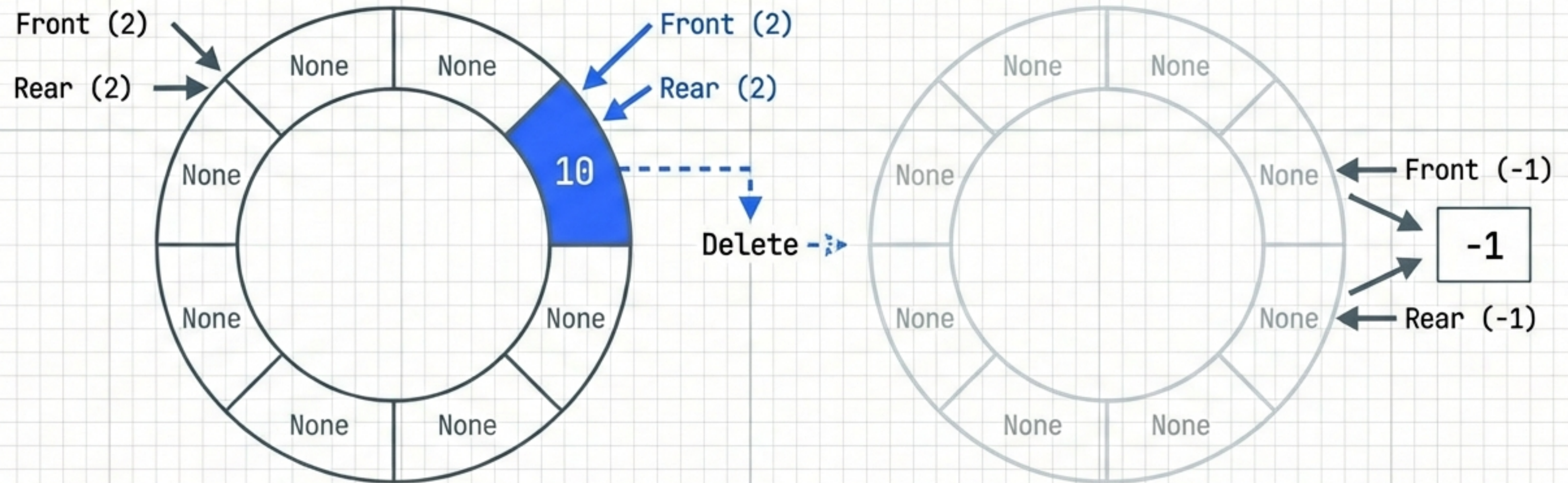


IS FULL: `(self.rear + 1) % size == self.front`

The “Single Element” Special Case

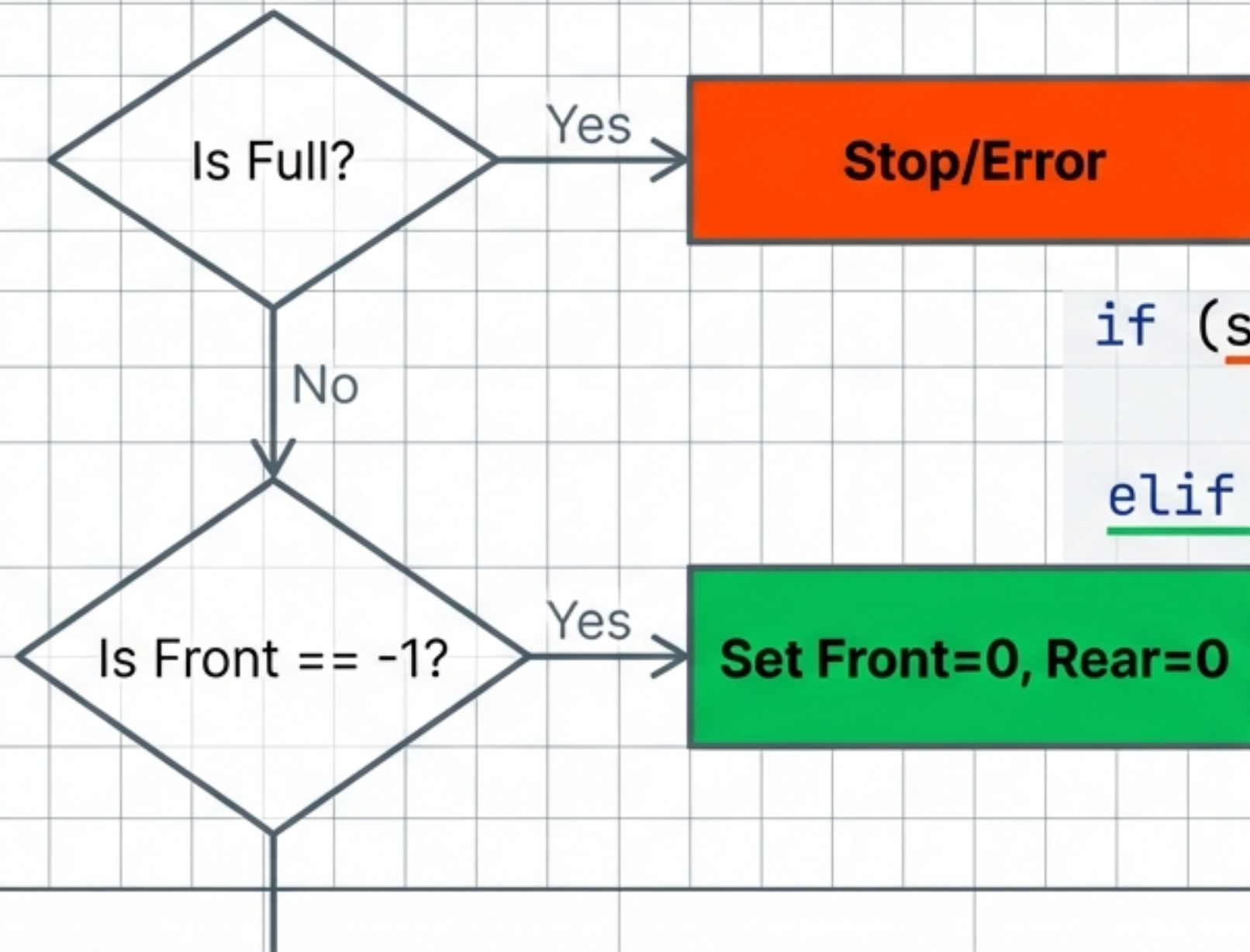
Condition: `if self.front == self.rear`

Action: If we delete this, we must reset the entire queue state (`front = -1, rear = -1`). Do not increment.



Enqueue: The Safety Checks

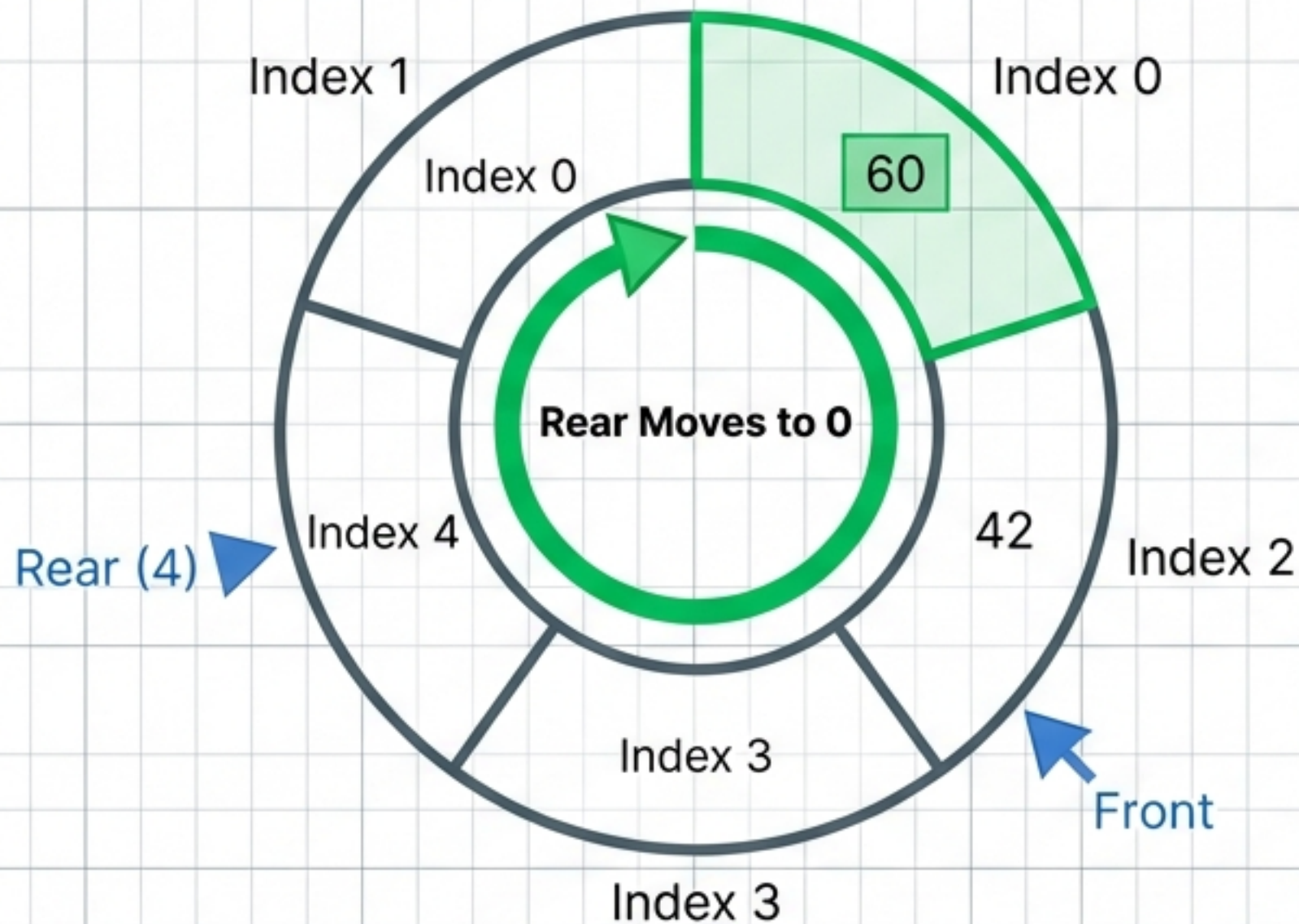
First, guard against overflow. If it's the very first element, manually drag pointers from -1 to 0.



```
if (self.rear + 1) % self.size == self.front:  
    print('Queue is Full')  
elif self.front == -1:  
    self.front = 0; self.rear = 0  
    self.items[self.rear] = value
```


Enqueue: The Cyclic Step

If space exists and it's not the first element, move Rear forward.
The Modulo operator handles the wrap-around automatically.

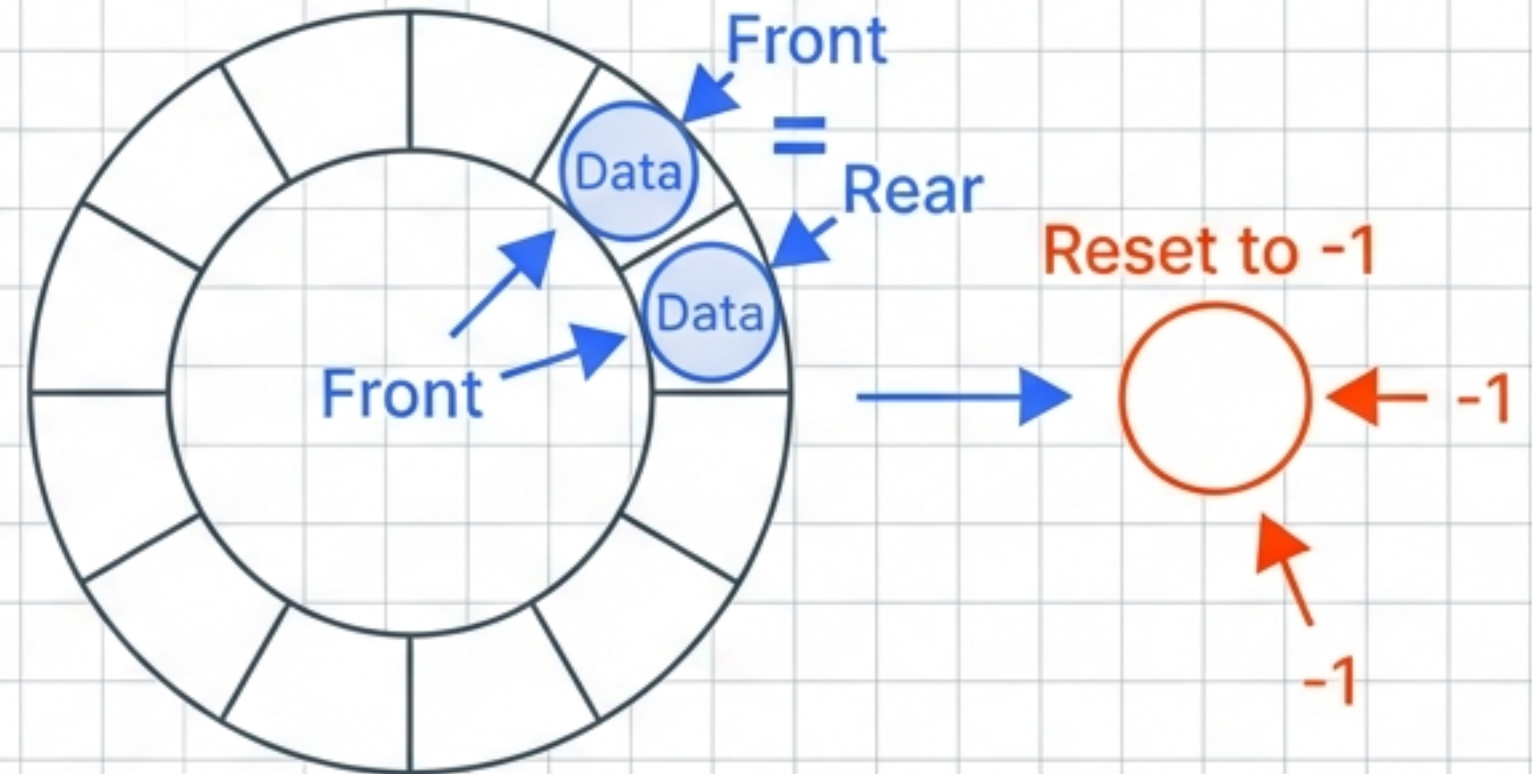


```
else:  
    self.rear = (self.rear + 1) % self.size  
    self.items[self.rear] = value
```


Deque: The Reset Logic

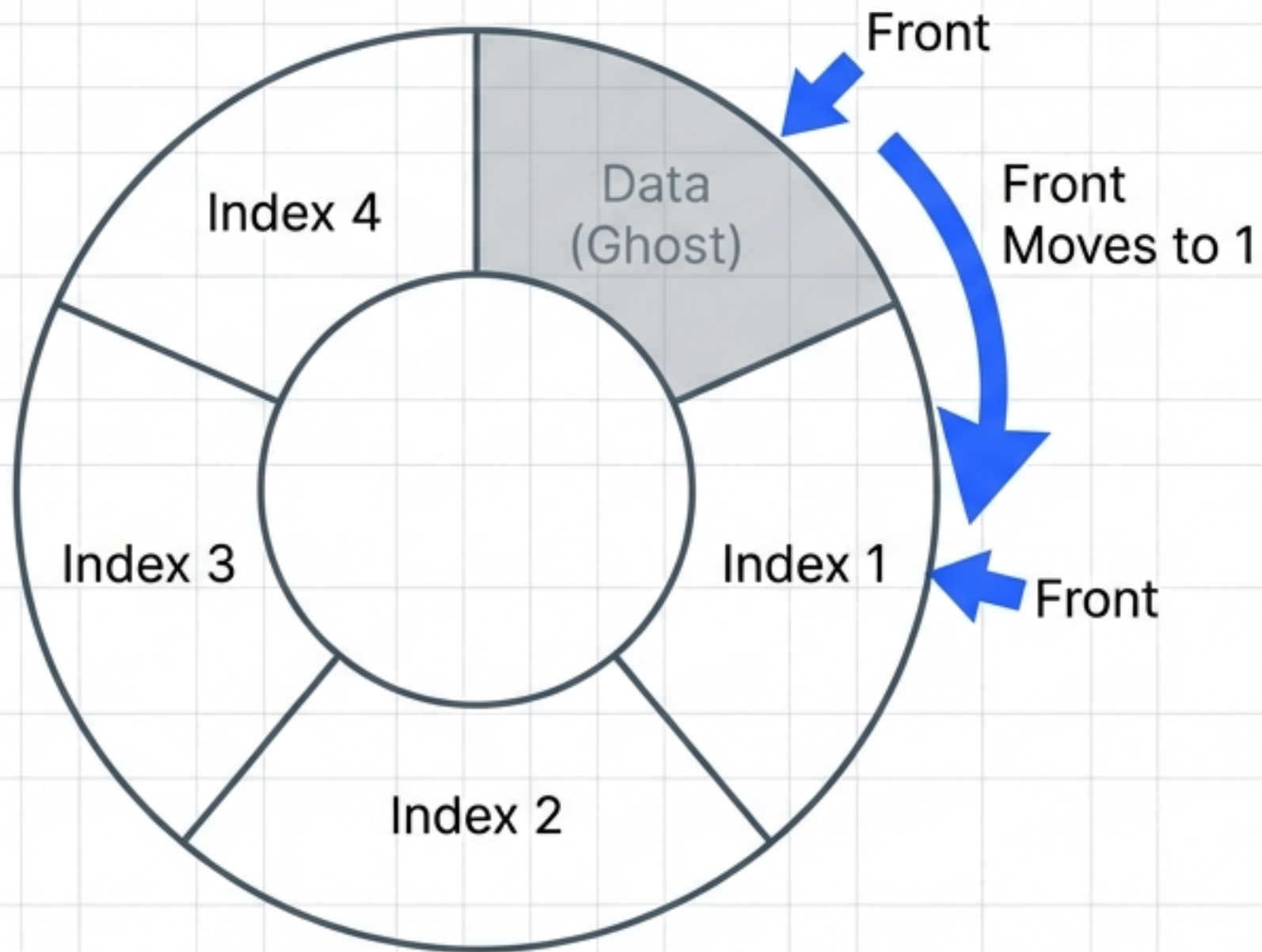
Crucial Step: If $\text{Front} == \text{Rear}$, we are deleting the last item. We must reset pointers to -1 to mark the queue as completely empty.

```
1 if self.front == -1:  
2     print('Queue is Empty')  
3 elif self.front == self.rear:  
4     self.front = -1; self.rear = -1
```



Deque: Moving Forward

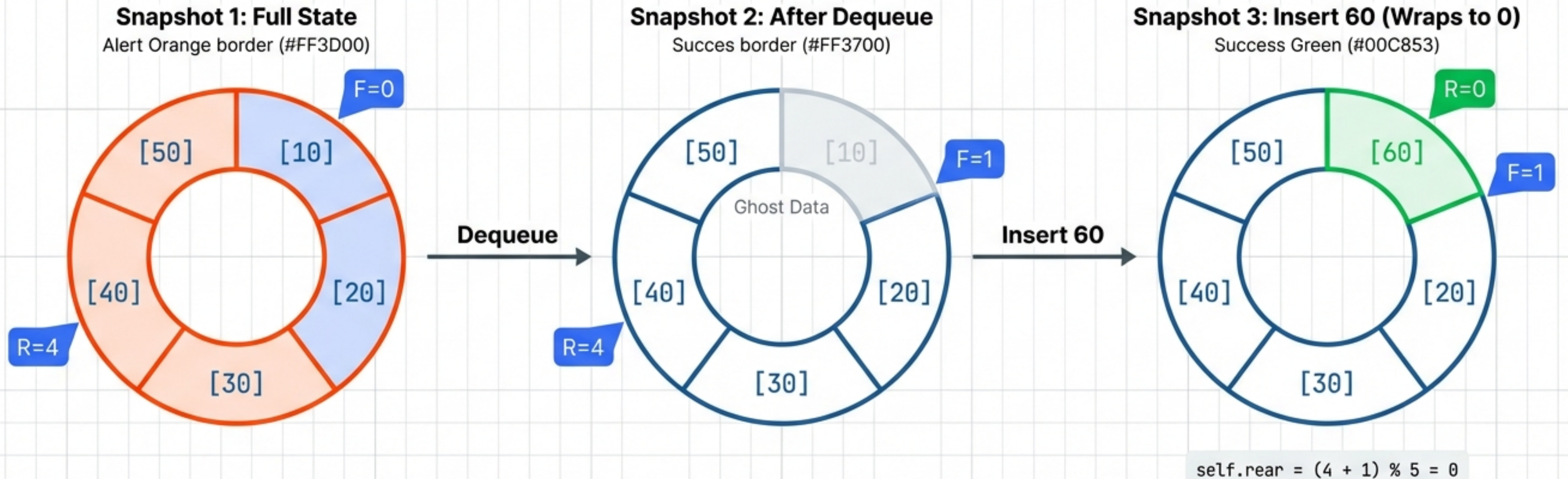
To 'delete', we simply move the Front pointer forward using Modulo. The previous data becomes "ghost" data, ready to be overwritten by the Rear later.



```
else:  
    self.front = (self.front + 1) % self.size
```


The Simulation Trace

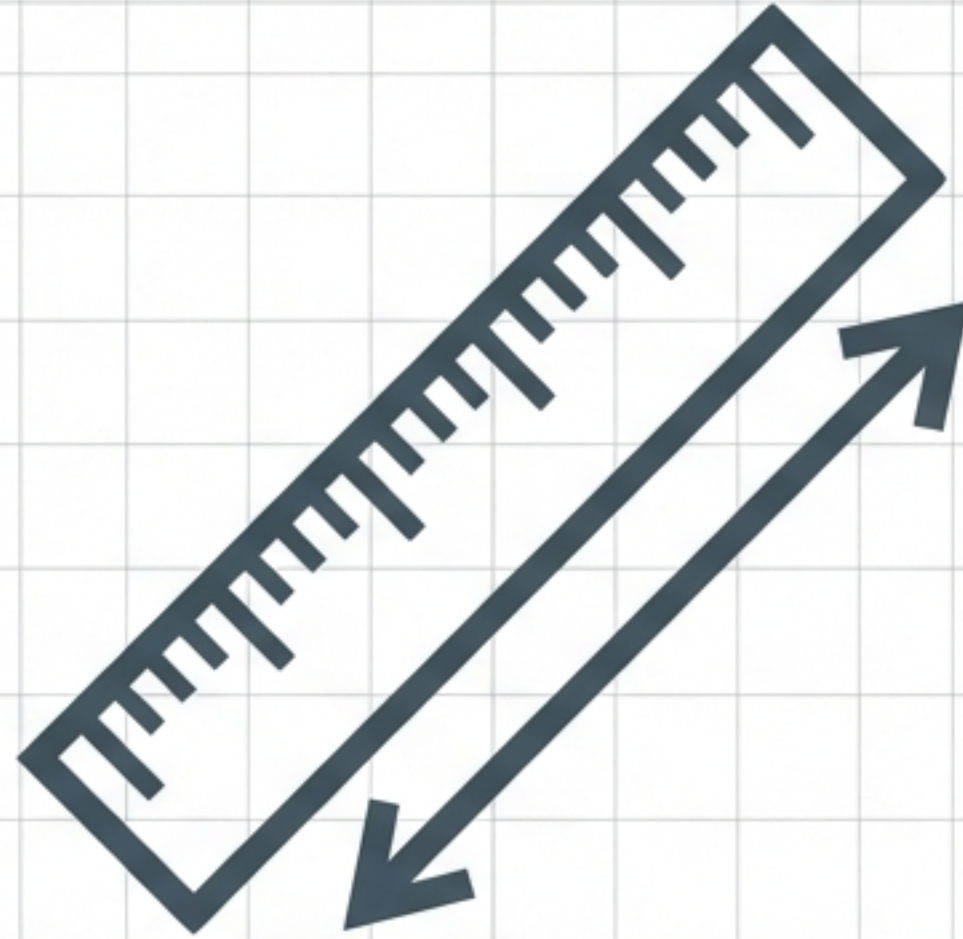
Proof of concept: In a linear queue, inserting 60 would fail.
Here, Rear wraps to 0 to utilize the space freed by the deletion of 10.



Pitfalls & Implementation Tips



Self Reference: Always use `self.front` and `self.rear`. Losing object scope is a common Python error.



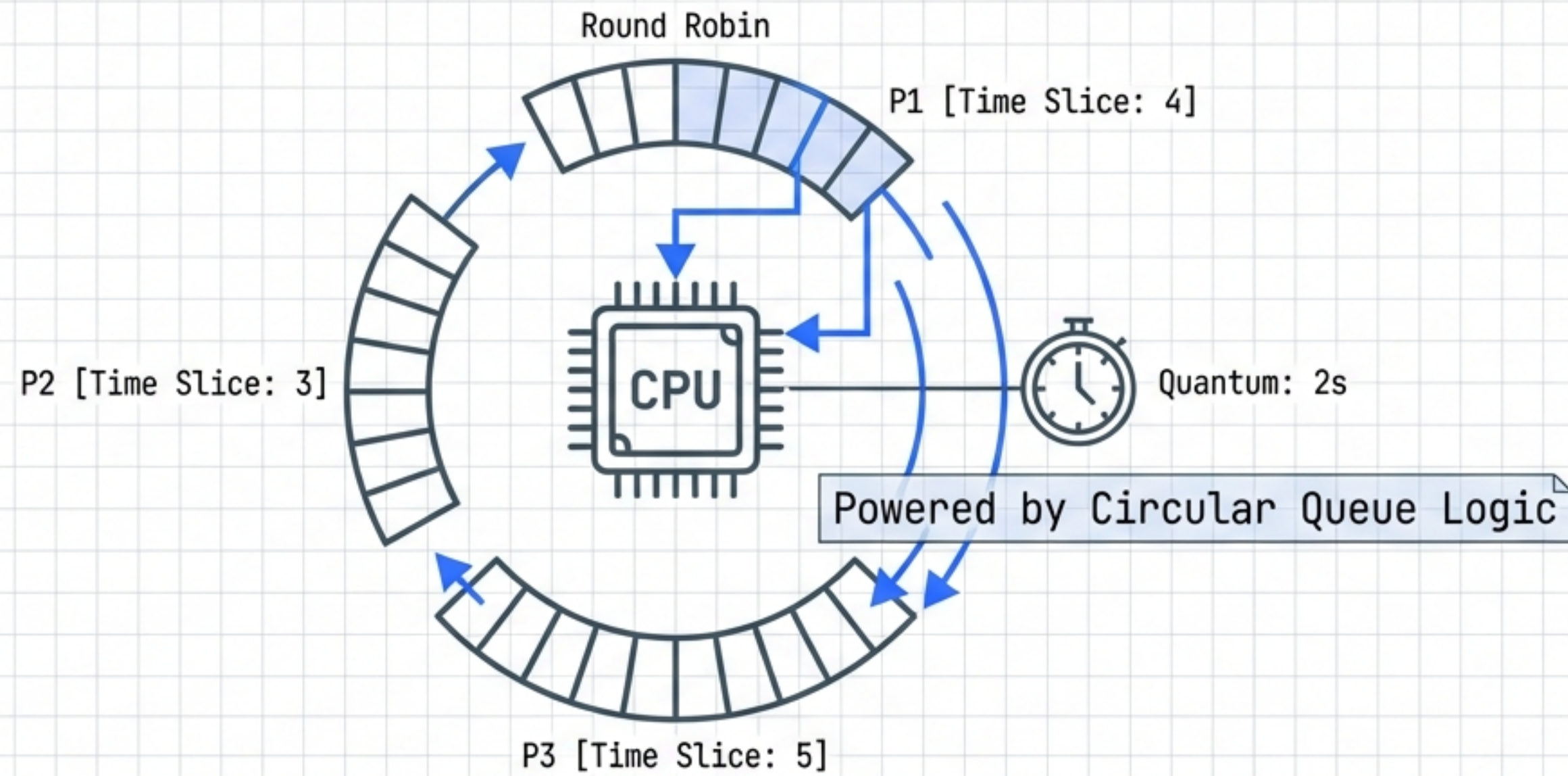
Modulo Size: Always modulo by `self.size`, never by the index itself.



Mental Model: Do not code blindly. Draw the **circle first**. Remember the **-1 reset**.

Beyond the Array

You have now implemented the core logic behind CPU Scheduling and streaming buffers.
Next Challenge: Implement a Deque (Double Ended Queue).



Code logic, not just syntax.